

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**BÁO CÁO MÔN HỌC
CÁC VẤN ĐỀ HIỆN ĐẠI CỦA CÔNG NGHỆ THÔNG TIN
Lớp: INT3507 7**

**ATTENTION-BASED CNN-LSTM AND
XGBOOST HYBRID MODEL FOR STOCK
PREDICTION**

Nhóm 12

Cao Trung Hiếu	20020403
Nguyễn Minh Quân	21020383
Nguyễn Thị Hoài Thu	22021135
Trần Minh Sơn	21020390

HÀ NỘI - 2024

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**

**BÁO CÁO MÔN HỌC
CÁC VẤN ĐỀ HIỆN ĐẠI CỦA CÔNG NGHỆ THÔNG TIN
Lớp: INT3507 7**

**ATTENTION-BASED CNN-LSTM AND
XGBOOST HYBRID MODEL FOR
STOCK PREDICTION**

Cán bộ hướng dẫn: TS.Nguyễn Đức Anh

Nhóm 12

Cao Trung Hiếu	20020403 (trưởng nhóm)
Nguyễn Minh Quân	21020383
Nguyễn Thị Hoài Thu	22021135
Trần Minh Sơn	21020390

HÀ NỘI - 2024

PHÂN CÔNG CÔNG VIỆC

Họ tên	MSSV	Phân công công việc	Phần trăm
Cao Trung Hiếu (nhóm trưởng)	20020403	Tìm hiểu bài báo. Tìm hiểu mã nguồn. Tìm hiểu bài báo của nhóm phản biện, tạo ra các câu hỏi cho nhóm phản biện. Chạy mã nguồn. Viết báo cáo các phần: II.2, II.3, II.7, VI Review các phần việc của các thành viên trong nhóm. Trả lời các câu hỏi của nhóm phản biện và cả lớp.	27%
Nguyễn Minh Quân	21020383	Tìm hiểu bài báo. Tìm hiểu bài báo của nhóm phản biện, tạo ra các câu hỏi cho nhóm phản biện. Tìm hiểu mã nguồn. Viết báo cáo các phần: II.4, II.6, III.3 Trả lời các câu hỏi của nhóm phản biện và cả lớp.	25%

		In báo cáo.	
Nguyễn Thị Hoài Thu	22021135	Tìm hiểu bài báo. Tóm tắt nội dung từ báo cáo vào các slide trình bày. Thiết kế slide gọn gàng, dễ nhìn, sử dụng hình ảnh minh họa, biểu đồ, sơ đồ. Tối ưu hóa bố cục trình bày. Trình bày slide. Trả lời các câu hỏi của nhóm phản biện và cả lớp. Viết báo cáo các phần: I, II.1, II.4, III.1 Chỉnh sửa định dạng báo cáo. Trả lời các câu hỏi của nhóm phản biện và cả lớp.	26%
Trần Minh Sơn	21020390	Tìm hiểu bài báo. Thiết kế một số slide. Tìm hiểu mã nguồn để vẽ biểu đồ, sơ đồ của hệ thống AttCLX. Viết báo cáo các phần:	22%

		III.2, III.4, IV, V Trả lời các câu hỏi của nhóm phản biện và cả lớp.	
--	--	---	--

TÓM TẮT

Tóm tắt:

Thị trường chứng khoán đóng vai trò quan trọng trong sự phát triển kinh tế. Do tính biến động phức tạp của thị trường chứng khoán, việc nghiên cứu và dự đoán sự thay đổi giá cổ phiếu có thể giúp nhà đầu tư tránh được rủi ro. Mô hình chuỗi thời gian truyền thống ARIMA không thể mô tả được tính phi tuyến và không đạt được kết quả khả quan trong dự đoán chứng khoán. Vì mạng nơ-ron có khả năng tổng quát phi tuyến mạnh mẽ, bài báo này đề xuất một mô hình kết hợp Attention-based CNN-LSTM và XGBoost để dự đoán giá cổ phiếu.

Mô hình được xây dựng trong bài báo tích hợp mô hình chuỗi thời gian, CNN với cơ chế Attention, LSTM, và XGBoost trong một mối quan hệ phi tuyến, nhằm cải thiện độ chính xác dự đoán. Mô hình có thể khai thác đầy đủ thông tin lịch sử của thị trường chứng khoán trong nhiều giai đoạn.

Dữ liệu chứng khoán trước tiên được tiền xử lý bằng ARIMA. Sau đó, kiến trúc học sâu được áp dụng trong khung làm việc "tiền huấn luyện - tinh chỉnh". Mô hình tiền huấn luyện là Attention-based CNN-LSTM dựa trên khung làm việc sequence-to-sequence. Mô hình trước tiên sử dụng mạng tích chập để trích xuất các đặc trưng sâu từ dữ liệu gốc, sau đó sử dụng mạng LSTM để khai thác các đặc trưng chuỗi thời gian dài hạn. Cuối cùng, mô hình XGBoost được sử dụng để tinh chỉnh.

Kết quả cho thấy mô hình kết hợp này hiệu quả hơn và đạt độ chính xác dự đoán tương đối cao, có thể giúp các nhà đầu tư hoặc tổ chức đưa ra quyết định, đạt được mục tiêu tăng lợi nhuận và tránh rủi ro. Mã nguồn có sẵn tại <https://github.com/zshicode/Attention-CLX-stock-prediction>.

Từ khóa: *Attention mechanism, Convolutional Neural Networks, Long Short-Term Memory, XGBoost, stock prediction*

LỜI CAM ĐOAN

Nhóm em xin cam đoan toàn bộ nội dung trong báo cáo là do nhóm tự nghiên cứu, phân tích và thực hiện dựa trên sự cố gắng và nỗ lực của cả nhóm cũng như sự giúp đỡ, hướng dẫn tận tình của giảng viên. Cam kết không xuất hiện tình trạng sao chép hay sử dụng các phần mềm trí tuệ nhân tạo như ChatGPT. Những tài liệu tham khảo được trích dẫn một cách đầy đủ đồng thời ghi rõ ràng về nguồn gốc theo quy định của nhà trường.

MỤC LỤC

PHÂN CÔNG CÔNG VIỆC.....	1
MỤC LỤC.....	6
DANH MỤC TỪ VIẾT TẮT.....	7
I. Giới thiệu.....	8
1. Giới thiệu chung.....	8
2. Giới thiệu tác giả.....	8
3. Phát biểu bài toán.....	9
II. Hệ thống AttCLX.....	10
1. Động lực.....	10
2. ARIMA model.....	10
3. Seq2Seq.....	13
4. ACNN.....	15
5. Hàm mất mát.....	19
6. Tổng hợp kiến trúc: AttCLX.....	21
III. Quá trình thực nghiệm.....	22
1. Giới thiệu phần mềm.....	22
2. Tập dữ liệu.....	23
3. Giới thiệu cấu trúc mã nguồn.....	24
4. Kết quả thực nghiệm của tác giả.....	28
IV. Ứng dụng của bài báo.....	29
V. Kết luận.....	29
VI. Phụ lục.....	31

DANH MỤC TỪ VIẾT TẮT

STT	Ký hiệu chữ viết tắt	Chữ viết đầy đủ
1	ACNN	Attention-based CNN
2	ADF	Augmented Dickey–Fuller test
3	ARIMA	Autoregressive Integrated Moving Average
4	AttCLX	Attention CNN LSTM XGBoost
5	CNN	Convolutional Neural Network
6	LSTM	Long short-term memory
7	Seq2Seq	Sequence - to - Sequence
8	XGBoost	eXtreme Gradient Boosting

I. Giới thiệu

1. Giới thiệu chung

Thị trường chứng khoán đóng vai trò quan trọng trong sự phát triển kinh tế. Mô hình được sử dụng phổ biến cho việc dự đoán giá chứng khoán là ARIMA, tuy nhiên có nhược điểm không mô tả được tính phi tuyến của dữ liệu chứng khoán. Với sự phát triển của neural networks ngoài ra có thể khắc phục được vấn đề trên cho nên các tác giả đã đề xuất một mô hình kết hợp Attention-based CNN-LSTM và XGBoost để dự đoán giá cổ phiếu.

Bài báo đưa ra một mô hình tích hợp mô hình ARIMA, CNN với cơ chế Attention, LSTM, và XGBoost trong một mối quan hệ phi tuyến, nhằm cải thiện độ chính xác của dự đoán. Từ đó có thể khai thác toàn diện thông tin lịch sử của thị trường chứng khoán qua nhiều giai đoạn.

Dữ liệu chứng khoán trước tiên được tiền xử lý thông qua ARIMA. Sau đó, kiến trúc học sâu được áp dụng theo khung làm việc pretraining-finetuning. Kiến trúc đó bao gồm 2 phần từ ACNN là LSTM. Cuối cùng, mô hình XGBoost được sử dụng để tinh chỉnh.

Mô hình này cho kết quả tốt hơn và xử lý được vấn đề của dữ liệu theo chuỗi thời gian.

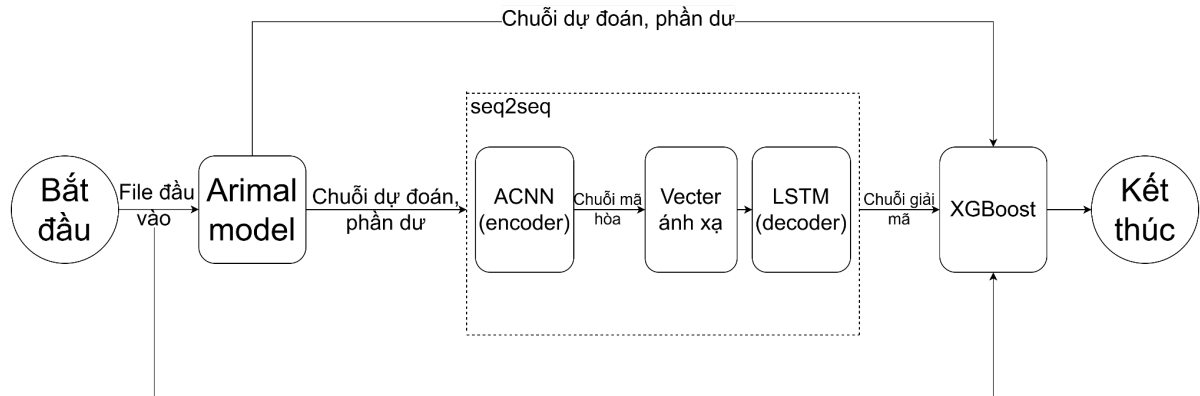
2. Giới thiệu tác giả

Công trình này được hỗ trợ bởi Orient Securities Co. Ltd và Dự án nghiên cứu kiểm soát Entropy của Shenzhen Wukong Investment Management Co. Ltd.

Tác giả:

- Zhuangwei Shi (email:zwshi@mail.nankai.edu.cn) và Yang Hu thuộc Khoa Trí tuệ nhân tạo, Đại học Nam Khai, Thiên Tân, Trung Quốc và Viện nghiên cứu Thâm Quyền, Đại học Nam Khai, Thâm Quyền, Quảng Đông, Trung Quốc.
- Guangliang Mo thuộc Công ty TNHH Chứng khoán Phương Đông, Thượng Hải, Trung Quốc.
- Jian Wu thuộc Trung tâm nghiên cứu Phát triển và Tài chính Vành đai kinh tế Châu Giang-Tây Giang (PWEB) và Viện Phát triển khu vực, Viện Khoa học xã hội Quảng Tây, Nam Ninh, Quảng Tây, Trung Quốc.

3. Phát biểu bài toán



3.1. Input

Dữ liệu chuỗi thời gian về giá cổ phiếu, bao gồm:

- Giá cổ phiếu trong từng ngày.
- Chuỗi thời gian đã được xử lý trước bằng mô hình ARIMA, gồm chuỗi sai phân bậc 1 và phần dư.

3.2. Output

Kết quả dự đoán giá cổ phiếu trong những ngày tiếp theo.

3.3. Vấn đề cần giải quyết

- Tính phi tuyến của dữ liệu chứng khoán.
- Đặc trưng của dữ liệu bao gồm đặc trưng cục bộ (ngắn hạn) và đặc trưng toàn cục (dài hạn).

3.4. Ý tưởng chính

- Sử dụng ARIMA để tiền xử lý dữ liệu.
- Sử dụng Attention-based CNN để trích xuất đặc trưng cục bộ
- LSTM để xử lý mối quan hệ chuỗi dài hạn
- XGBoost để tinh chỉnh kết quả nhằm tăng độ chính xác của dự đoán.

3.5. Các bước thực nghiệm chính

- Tiền xử lý dữ liệu (Preprocessing)
- Mô hình tiền huấn luyện (Pretraining)

- Tinh chỉnh (Fine-tuning)
- Đánh giá và so sánh hiệu quả
- Thực nghiệm và kết quả

II. Hệ thống AttCLX

1. Động lực

1.1. Hạn chế của phương pháp truyền thống

- **ARIMA**: Chuỗi dữ liệu đầu vào phải là chuỗi tuyến tính. Tuy nhiên đặc điểm của dữ liệu chứng khoán là chuỗi phi tuyến do phụ thuộc vào nhiều yếu tố bên ngoài.
- Mối quan hệ toàn cục và cục bộ không được xử lý hiệu quả.

1.2. Sự phát triển của trí tuệ nhân tạo (AI)

- **Tiềm năng của các mạng học sâu (Deep Learning)**: Các mạng nơ-ron có khả năng tổng quát hóa phi tuyến mạnh mẽ, giúp giải quyết hạn chế của ARIMA.
- **Kết hợp Attention Mechanism và LSTM**: Các cơ chế Attention có thể mô phỏng khả năng tập trung của con người vào thông tin quan trọng và LSTM mô hình hóa tốt hơn các phụ thuộc dài hạn trong dữ liệu.

1.3. Mục tiêu của bài báo

- Đề xuất một mô hình lai kết hợp ARIMA với các mạng học sâu tiên tiến để cải thiện độ chính xác của dự đoán giá cổ phiếu.

2. ARIMA model

2.1. Khái niệm:

Mô hình kiến trúc: ARIMA (AutoRegressive Integrated Moving Average) là một mô hình dự báo chuỗi thời gian, thường được dùng để dự báo giá cổ phiếu hoặc các loại tài sản biến động theo thời gian. Cấu trúc của ARIMA gồm 3 phần chính:

- **AutoRegressive (AR)**: phần tự hồi quy, thể hiện quan hệ giữa giá trị ở thời điểm hiện tại của chuỗi và p giá trị trong quá khứ.
- **Integrated (I)**: phần tích hợp, sử dụng sai phân bậc d để biến đổi các giá trị sao cho chuỗi các giá trị tạo thành chuỗi dừng (stationary).

- **Moving Average (MA):** phần trung bình động, thể hiện rằng các giá trị luôn có xu hướng hướng về đường trung bình động của q giá trị liên nhau.

Công thức của mô hình ARIMA với các tham số p, d, q:

$$s_t = a_0 + \sum_{i=1}^p a_i s_{t-i} + w_t + \sum_{i=1}^q b_i w_{t-i}$$

Trong đó:

- s_t : Giá trị của chuỗi thời gian tại thời điểm t (Ví dụ: giá trị của giá cổ phiếu tại thời điểm t).
- p, d, q: các tham số của mô hình.
- a_i và b_i : các hệ số của mô hình tự hồi quy và trung bình động.
- w_t : biến ngẫu nhiên được lấy từ phân phối chuẩn (Normal distribution).

Điều kiện để ARIMA hoạt động:

- Chuỗi giá trị $s_t, s_{t-1}, \dots, s_0$ phải là chuỗi dừng, nghĩa là các giá trị đó được lấy ra từ cùng một phân phối xác suất hoặc đảm bảo hai điều kiện:
 - $E[s_t] = E[s_{t-1}] = \dots = E[s_0]$: giá trị trung bình của các giá trị trong chuỗi là không đổi.
 - $\text{Var}[s_t] = \text{Var}[s_{t-1}] = \dots = \text{Var}[s_0]$: phương sai của các giá trị trong chuỗi là không đổi.

2.2. Ưu điểm và nhược điểm của việc sử dụng ARIMA:

2.2.1. Ưu điểm:

Trong bài nghiên cứu ARIMA được sử dụng để dự đoán giá trị ban đầu trong tương lai, làm đầu vào cho mô hình AttCLX, nó giúp loại bỏ các yếu tố gây nhiễu và tạo ra xu hướng của dữ liệu một cách chính xác nhất.

2.2.2. Nhược điểm:

ARIMA không thể tạo ra các dự đoán chính xác với các chuỗi phi tuyến tính và có thể gây sai lệch nghiêm trọng nếu sử dụng ARIMA để dự đoán một chuỗi phi tuyến tính

2.3. Vai trò của ARIMA trong kiến trúc của mô hình AttCLX:

ARIMA đóng vai trò là bước tiền xử lý dữ liệu đầu vào của mô hình AttCLX. ARIMA giúp kiểm tra một mã cổ phiếu có thể dự đoán được hay không bằng cách kiểm tra giá đóng cửa của cổ phiếu đó có phải là chuỗi dừng, nếu giá đóng cửa không phải là một chuỗi dừng thì lấy sai phân bậc một giữa hai giá cổ phiếu liên nhau. Quá trình này bao gồm ba giai đoạn chính:

- **Phân tích độ dừng:** các mô hình ARIMA sử dụng bài test ADF (Augmented Dickey–Fuller test) để kiểm tra chuỗi đầu vào có phải là chuỗi dừng hay không (giá cổ phiếu theo thời gian). Nếu chuỗi không phải là một chuỗi dừng, tác giả thực hiện lấy sai phân bậc một và thực hiện kiểm tra lại một lần nữa bằng ADF.
- **Thiết lập các tham số của mô hình:** tác giả sử dụng hai hàm ACF (Autocorrelation function) và PACF (Partial autocorrelation) để xác định hai tham số p và q trong mô hình ARIMA và nhận thấy giá cổ phiếu đang được xét phù hợp với $p = 2$ và $q = 0$.
- **Tạo dựng chuỗi giá trị của mô hình:** sử dụng công thức của mô hình ARIMA ($p=2, q=0, d=1$) tác giả tạo ra chuỗi giá trị tương ứng với mô hình, lấy các giá trị đó làm đầu vào cho mô hình AttCLX.

2.4. Cách kiểm tra chuỗi dừng bằng bài kiểm tra Augmented Dickey-Fuller Test (ADF test):

2.4.1. Giả thuyết trong ADF test:

- Giả thuyết gốc H_0 : chuỗi có đơn vị gốc, nghĩa là chuỗi không dừng.
- Giả thuyết đối H_1 : chuỗi dừng (không có đơn vị gốc).

2.4.2. Công thức của ADF test:

$$\Delta y_t = \alpha + \gamma * y_{t-1} + \varepsilon_t$$

Trong đó:

- y_t : giá cổ phiếu của ngày thứ t .
- $\Delta y_t = y_t - y_{t-1}$: sai phân bậc 1 của chuỗi y_t
- α : hằng số.
- γ : hệ số của y_{t-1} và là thành phần quan trọng quyết định tính dừng của chuỗi.
- ε_t : sai số ngẫu nhiên (nhiều trắng) được lấy từ phân phối chuẩn $N(0, \sigma^2)$.

Giá trị của γ quyết định kết quả của kiểm định:

- Nếu $\gamma = 0$ thì chuỗi có đơn vị gốc (không dừng).
- Nếu $\gamma < 0$ thì chuỗi là chuỗi dừng.

2.4.3. Giải thích về hệ số $\gamma = 0$:

Nếu $\gamma = 0$: phương trình trở thành:

$$\Delta y_t = \alpha + \varepsilon_t$$

$$\Leftrightarrow y_t - y_{t-1} = \alpha + \varepsilon_t$$

$$\Leftrightarrow y_t = y_{t-1} + \alpha + \varepsilon_t$$

$$\Leftrightarrow y_t = y_0 + \alpha + \sum_{i=1}^t \varepsilon_i$$

$$\Leftrightarrow E[y_t] = E[y_0] + \alpha = \mu \text{ (vì } E[\varepsilon_i] = 0)$$

$$\Leftrightarrow \text{Var}(y_t) = \text{Var}(y_0 + \alpha + \sum_{i=1}^t \varepsilon_i) = t\sigma^2$$

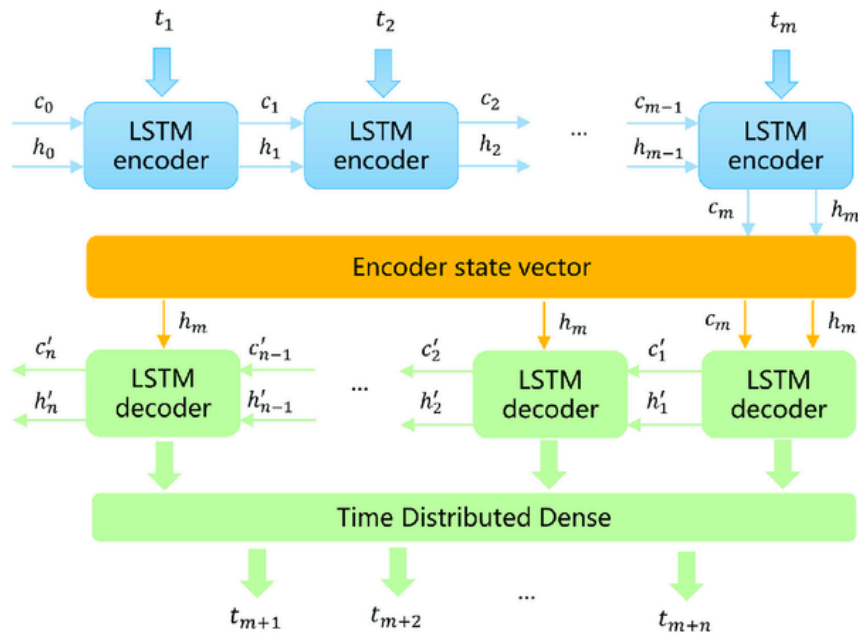
$\Rightarrow$ Tuy giá trị trung bình của chuỗi không đổi (bằng μ) nhưng phương sai của y_t không phải là một hằng số và được tích lũy qua thời gian (thời gian càng lớn thì phương sai càng lớn) do đó chuỗi không dừng khi $\gamma = 0$.

3. Seq2Seq

3.1. Khái niệm:

Seq2Seq (Sequence - to - Sequence) là một mô hình bao gồm hai phần chính: bộ mã hóa (encoder) và bộ giải mã (decoder) và được sử dụng trong các bài toán có giá trị đầu vào là chuỗi giá trị theo thời gian và đầu ra là một chuỗi có thể có định dạng khác.

- **Bộ mã hóa (Encoder):** chuỗi đầu vào được mã hóa thành các ngữ cảnh và trạng thái ẩn.
- **Bộ giải mã (Decoder):** đầu vào của decoder là trạng thái ẩn và ngữ cảnh từ encoder, decoder tạo ra chuỗi đầu ra, giải mã trạng thái ẩn kết hợp với vector ngữ cảnh để tạo thành các giá trị của chuỗi đầu ra.



Kiến trúc của Seq2Seq.

(Nguồn:

https://www.researchgate.net/figure/Illustration-of-Seq2Seq-LSTM-architecture_fig2_367260783?__cf_chl=tk=EOZU6i6A9kRE__9i0n3EpYUwoXo9EzP2SQLbjCC9Ok4-1733910914-1.0.1.1-kdNeMcKEc0OroW_qjTtCDxH.Bh3SQT6pUxLEBDhp84w)

Seq2Seq được sử dụng trong các bài toán cần xử lý dữ liệu có chiều dài không cố định hoặc các bài toán chuỗi đầu vào và chuỗi đầu ra có dạng khác nhau, như dự đoán giá cổ phiếu, dịch câu hoặc đoạn văn.

Công thức: một Seq2Seq sử dụng LSTM làm encoder và decoder. Chuỗi đầu vào là $X=(x_1, x_2, \dots, x_n)$ và chuỗi đầu ra là $Y=(y_1, y_2, \dots, y_m)$. Trạng thái ẩn của encoder tại thời điểm t là h_t , giá trị này được tính toán dựa trên trạng thái trước đó h_{t-1} và đầu vào hiện tại x_t :

$$h_t = \text{LSTM}(x_t, h_{t-1})$$

Sau khi hoàn thành chuỗi đầu vào, trạng thái cuối cùng của encoder sẽ được truyền tới decoder. Decoder dự đoán từng bước đầu ra y_t dựa trên giá trị đầu ra trước đó:

$$y_t = \text{LSTM}(y_{t-1}, h_{t-1})$$

3.2. Ưu điểm và nhược điểm của việc sử dụng Seq2seq:

3.2.1. Ưu điểm:

Seq2Seq cho phép xử lý dữ liệu đầu vào dạng chuỗi và đầu ra là một chuỗi có thể có dạng khác với đầu vào. Trong bài nghiên cứu, mô hình Seq2Seq được sử dụng để lấy các đặc trưng từ chuỗi thông tin giá cổ phiếu, mô hình có thể cải thiện khả năng dự báo của hệ thống khi kết hợp với các cơ chế khác như cơ chế chú ý (Attention).

3.2.2. Nhược điểm:

Mô hình Seq2Seq gặp vấn đề phụ thuộc xa khi chuỗi đầu vào là một chuỗi dài, điều này làm mất mát thông tin khi dữ liệu đi qua các lớp trong encoder. Để giải quyết, bài báo kết hợp Seq2Seq với cơ chế chú ý (Attention) để tạo ra các trọng số với từng đầu vào, từ đó xác định được phần nào của đầu vào là quan trọng đối với đầu ra, giúp giảm thiểu việc mất mát thông tin.

3.3. Vai trò của Seq2seq trong kiến trúc của mô hình AttCLX:

Trong bài báo, mô hình Seq2Seq đóng góp các vai trò sau trong kiến trúc:

- **Mã hóa và giải mã dữ liệu giá cổ phiếu với đầu vào là dữ liệu do mô hình ARIMA dự đoán:** Seq2Seq trích xuất các đặc trưng từ dữ liệu do mô hình ARIMA dự đoán bằng encoder sau đó giải mã các chuỗi đặc trưng này bằng decoder để có thể tạo ra một chuỗi phi tuyến tính gần giống nhất với giá cổ phiếu thực tế.
- **Kết hợp Attention và CNN (ACNN):** Seq2Seq khi được kết hợp với ACNN sẽ cải thiện được vấn đề phụ thuộc xa, mô hình có thể tập chung vào các điểm dữ liệu có đặc trưng nhất nhờ cơ chế chú ý (Attention) và có thể trích xuất được các đặc trưng của dữ liệu nhờ CNN.

4. ACNN

4.1. Attention mechanism

4.1.1. Khái niệm cơ bản về Attention

- Attention là một kỹ thuật trong học sâu dựa trên cơ chế nhận thức của con người.
- Khi xử lý dữ liệu đầu vào, con người không quan sát toàn bộ dữ liệu cùng lúc mà chỉ tập trung vào những phần quan trọng nhất.
- Attention mô phỏng cách làm này, cho phép mô hình học tập chọn lọc những phần thông tin có quan trọng trong một chuỗi dữ liệu.

4.1.2. Công thức của Attention

- Đầu vào của Attention bao gồm một chuỗi dữ liệu $X=(x_1, x_2, \dots, x_n)$, và một vector truy vấn q . Attention tính toán điểm số (score) giữa các thành phần của chuỗi dữ liệu x_i và vector truy vấn q , thông qua một hàm tính điểm $s(x_i, q)$. Công thức:

$$\alpha_i = p(z = i | X, q) = \frac{\exp(s(x_i, q))}{\sum_{j=1}^N \exp(s(x_j, q))}. \quad (4)$$

- Trong đó, α_i là trọng số của phần tử x_i dựa trên mức độ liên quan của nó với q (truy vấn). Điểm số được chuẩn hóa bằng hàm **softmax**, đảm bảo tổng các trọng số bằng 1.
- Hàm tính điểm (score) $s(x_i, q)$ được sử dụng trong mô hình thường là **dot-product** (tích vô hướng) giữa x_i và q , sau đó chia cho căn bậc hai của kích thước vector d :

$$s(x_i, q) = \frac{x_i^T q}{\sqrt{d}}$$

Đây là công thức của **scaled dot-product attention**, một trong những kỹ thuật tính toán phổ biến trong Attention.

4.1.3. Multi-head Attention

- Để tăng cường khả năng học tập, cơ chế **Multi-head Attention** được áp dụng. Thay vì chỉ sử dụng một vector truy vấn q , Multi-head Attention sử dụng nhiều vector truy vấn cùng lúc $Q=[q_1, q_2, \dots, q_M]$. Các đầu (heads) tính toán Attention song song cho mỗi truy vấn q_i , sau đó kết hợp kết quả lại với nhau để có đầu ra cuối cùng:

$$att((K, V), Q) = [att((K, V), q_1) || att((K, V), q_2) || \dots || att((K, V), q_M)]$$

4.1.4. Self-Attention

- Trong Self-Attention, các vector truy vấn Q , khóa K và giá trị V (key, value) đều được sinh ra từ chính dữ liệu đầu vào X thông qua các phép biến đổi tuyến tính:

$$Q = W_Q X, K = W_K X, V = W_V X. \quad (9)$$

- Kết quả đầu ra của Self-Attention được tính như sau:

$$H = V \text{softmax} \left(\frac{K^T Q}{\sqrt{d}} \right)$$

4.1.5. Ưu điểm

- Tập trung vào thông tin quan trọng và bỏ qua các thông tin gây nhiễu
- Attention cơ bản giải quyết được các hạn chế trong LSTM liên quan đến phụ thuộc dài hạn, nhờ vào khả năng gán trọng số cho các bước thời gian khác nhau.

4.1.6. Nhược điểm

- **Chi phí tính toán cao**
 - Attention, đặc biệt là self-attention, đòi hỏi tài nguyên tính toán lớn, đặc biệt khi xử lý các chuỗi thời gian dài.
- **Dễ bị overfitting**
 - Với các mô hình nhỏ hoặc dữ liệu không đủ, cơ chế Attention có thể làm mô hình tập trung quá mức vào một số đặc trưng không đáng kể, dẫn đến hiện tượng overfitting.

4.1.7. Ứng dụng của Attention trong mô hình chuỗi thời gian

- Attention có thể được áp dụng để tạo ra trọng số dữ liệu, cho phép mô hình tập trung vào các phần quan trọng hơn của chuỗi dữ liệu. **Attention-based Convolutional Neural Networks (ACNN)** kết hợp với LSTM để khai thác các mối quan hệ dài hạn và ngắn hạn trong dữ liệu chuỗi thời gian.
- Sự kết hợp giữa Attention và LSTM giúp mô hình có khả năng phân tích cả các mối quan hệ toàn cục (global) và cục bộ (local) trong chuỗi, cải thiện hiệu suất dự đoán.

4.2. CNN

4.2.1. Khái niệm

CNN (Convolutional Neural Network) là một hệ thống mạng nơ-ron tích chập cho phép người dùng xây dựng hệ thống thông tin với độ chính xác cao, thường được sử dụng trong các tác vụ liên quan đến nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên, và phân tích chuỗi thời gian. Cơ chế tích chập của CNN giúp khả năng tự học các đặc trưng từ dữ liệu dạng lưới (như hình ảnh hoặc âm thanh) một cách hiệu quả mà không cần phải sử dụng phương pháp trích xuất đặc trưng thủ công.

Trong bài toán này, CNN đóng vai trò là bộ encoder trong mô hình seq2seq với đầu vào là 1 chuỗi giá trị nên ta sử dụng biến thể CNN 1 chiều (CNN 1D) để xử lý chuỗi dữ liệu này.

Trong CNN, việc trích xuất các đặc trưng diễn ra qua các lớp tích chập (convolutional layers). Tuy nhiên, sự kết hợp của Attention cho phép CNN mở rộng khả năng của mình bằng cách gán trọng số cho các phần khác nhau của dữ liệu dựa trên tầm quan trọng của chúng. CNN không chỉ xử lý đặc trưng cục bộ mà còn nắm bắt mối quan hệ toàn cục (global dependencies) trong dữ liệu chuỗi.

4.2.2. Cách hoạt động của CNN một chiều

- Tích chập một chiều:
 - Tạo Feature map bằng cách cho lớp Kernel di chuyển dọc chuỗi dữ liệu và tính toán tích vô hướng của kernel và đoạn dữ liệu chạy qua.
 - Tích hợp các bộ lọc có kích thước khác nhau để học các mẫu ngắn hạn và dài hạn.
- Kết hợp với Attention: Áp dụng cơ chế Self-attention để tính các trọng số ảnh hưởng của giá trị đầu ra.

4.2.3. Lợi ích và vai trò của CNN

- Vai trò của CNN:
 - Trích xuất đặc trưng từ chuỗi thời gian: các tầng tích chập giúp mô hình nắm bắt các biến động của giá cổ phiếu hiệu quả.
 - Sự kết hợp của self-attention và multi-head attention trong mô hình giúp nắm bắt chính xác các mối quan hệ dài hạn trong chuỗi.
 - CNN đóng vai trò là encoder trong mô hình seq2seq: xử lý chuỗi thời gian và tạo các đặc trưng đầu ra để truyền cho decoder (LSTM).
- Lợi ích của CNN:
 - Nắm bắt tốt cả các đặc trưng cục bộ và toàn cục.
 - Cơ chế attention giúp giảm nhiễu và tập chung vào các thông tin cần thiết.
 - Kết hợp hiệu quả với LSTM.

5. Hàm mất mát

5.1. Khái niệm

Hàm mất mát (Loss Function) là các hàm toán học được sử dụng trong học máy để tính toán các số liệu với mục đích so sánh sự khác biệt giữa giá trị dự đoán của mô hình và giá trị thực tế. Kết quả của các Hàm mất mát giúp đánh giá hiệu suất của mô hình trong quá trình huấn luyện, từ đó đưa ra phương hướng tối ưu hóa tham số của mô hình.

Trong bài báo, có 4 công thức được sử dụng là MAE, RMSE, MAPE và R². Trong đó, hàm RMAE được sử dụng để điều chỉnh các tham số trong công thức của mô hình

5.2. Các công thức được sử dụng

Trong bài báo này, nhóm tác giả đã sử dụng nhiều hàm mất mát khác nhau để đánh giá hiệu suất của mô hình, bao gồm:

1. Mean Absolute Error (MAE):

$$MAE = \frac{1}{n} \sum_{t=1}^n |\hat{X}_t - X_t|$$

- MAE đo lường sai số tuyệt đối trung bình giữa giá trị dự đoán và giá trị thực tế.

2. Root Mean Square Error (RMSE):

$$RMSE = \sqrt{\frac{1}{n} \sum_{t=1}^n (\hat{X}_t - X_t)^2}$$

- RMSE là căn bậc hai của sai số bình phương trung bình, nhấn mạnh các sai số lớn hơn trong quá trình đánh giá.

3. Mean Absolute Percentage Error (MAPE):

$$MAPE = \frac{1}{n} \sum_{t=1}^n \left| \frac{\hat{X}_t - X_t}{X_t} \right|$$

- MAPE đo lường sai số theo tỷ lệ phần trăm, phản ánh sai số của mô hình so với giá trị thực tế.

4. R-squared (R^2):

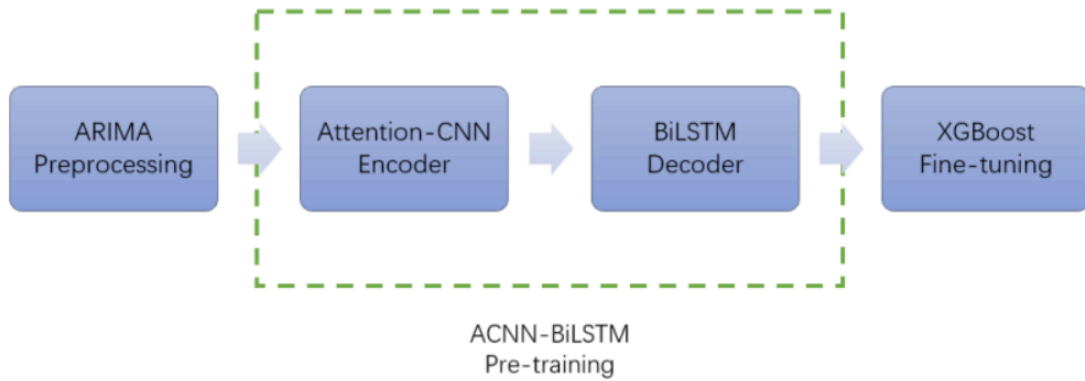
$$R^2 = \frac{\sum_{t=1}^n \|\hat{X}_t - \bar{X}_t\|^2}{\sum_{t=1}^n \|X_t - \bar{X}_t\|^2}$$

- R^2 là hệ số xác định, cho biết tỷ lệ phương sai trong dữ liệu thực tế được giải thích bởi mô hình.

5.3. Lợi ích và vai trò của Hàm mất mát

- Lợi ích:
 - Đánh giá hiệu suất: Giá trị của hàm mất mát cung cấp thông tin rõ ràng về độ chính xác của mô hình trong việc dự đoán giá trị, giúp xác định mức độ cải thiện.
 - Hướng dẫn tối ưu hóa: Hàm mất mát cho phép các thuật toán tối ưu hóa như Gradient Descent điều chỉnh tham số của mô hình để giảm thiểu sai số.
 - Phát hiện vấn đề: Theo dõi giá trị của hàm mất mát có thể phát hiện các vấn đề như hiện tượng quá khớp (overfitting) hoặc quá giản lược (underfitting) của mô hình.
- Vai trò: Hàm mất mát có vai trò đánh giá hiệu năng dự đoán chuỗi dữ liệu của mô hình Attention-based CNN-LSTM và XGBoost, giúp xác định chênh lệch giữa giá dự đoán và giá thực tế, từ đó giúp người phát triển điều chỉnh các tham số sao cho tối ưu và hiệu quả cao nhất có thể, cùng với đó là so sánh hiệu quả với các mô hình truyền thống khác.

6. Tổng hợp kiến trúc: AttCLX



Kiến trúc của AttCLX

Kiến trúc của AttCLX bao gồm 3 phần chính: tiền xử lý dữ liệu (preprocessing), training model, fine-tuning.

6.1. Tiền xử lý dữ liệu (preprocessing)

Dữ liệu gốc ban đầu là bốn loại giá cổ phiếu theo từng ngày (giá đóng cửa, giá mở cửa, giá cao nhất trong ngày, giá thấp nhất trong ngày), khối lượng giao dịch, số lượng giao dịch. Sau đó sử dụng mô hình ARIMA($p = 2, q = 0, d = 1$) để tạo ra chuỗi tuyến tính tương ứng với giá đóng cửa theo thời gian, công thức của ARIMA($p = 2, q = 0, d = 1$):

$$s_t = a_0 + a_1 s_{t-1} + a_2 s_{t-2} + w_t + b_1 w_{t-1} + b_2 w_{t-2}$$

Trong đó:

- s_t = (giá cổ phiếu ngày thứ t) - (giá cổ phiếu ngày thứ $t-1$)
- a_i và b_i : các hệ số của mô hình tự hồi quy và trung bình động.
- w_t : biến ngẫu nhiên được lấy từ phân phối chuẩn (Normal distribution).

Dữ liệu đầu ra của bước tiền xử lý dữ liệu sẽ gồm ba thành phần (features):

- Giá đóng cửa: c_t
- Giá trị s_t tương ứng được sinh ra bởi mô hình ARIMA($p = 2, q = 0, d = 1$)

- $r_t = p_t - s_t$, trong đó p_t là giá đóng cửa của ngày thứ t , r_t là giá trị khác nhau giữa giá trị thật của giá đóng cửa và giá trị dự đoán của mô hình ARIMA($p = 2, q = 0, d = 1$) ngày thứ t .

6.2. Training model

Model được sử dụng ở đây là Seq2Seq, với encoder là Attention-based CNN-LSTM (ACNN-LSTM) và decoder là Bidirectional LSTM. Dữ liệu từ bước tiền xử lý trước tiên được đi qua CNN một chiều để trích xuất các đặc tính quan trọng của dữ liệu, sau đó sử dụng LSTM để khai thác các đặc điểm dựa trên chuỗi thời gian dài của chuỗi dữ liệu. ACNN có thể trích xuất được các đặc trưng mang tính toàn cục và cục bộ của dữ liệu, điều mà LSTM có thể không làm được. Seq2Seq giúp loại bỏ các dữ liệu nhiễu trong chuỗi dữ liệu.

6.3. Fine-tuning model

Sau khi dữ liệu được decoder giải mã và trở thành đầu ra thì sẽ được đi qua XGBoost, XGBoost sẽ khai thác dữ liệu ở các chu kỳ khác nhau của dữ liệu (Ví dụ: lấy 20 ngày trước đó của giá cổ phiếu để dự đoán cho 1 ngày trong tương lai). XGBoost có khả năng kết hợp nhiều tree models để có thể tạo ra một model mạnh mẽ hơn.

III. Quá trình thực nghiệm

1. Giới thiệu phần mềm

Các bước chính trong quá trình thực nghiệm gồm:

- **Dữ liệu và cài đặt thực nghiệm:**
 - Dữ liệu được lấy từ **Tushare**, một nền tảng dữ liệu công khai về thị trường chứng khoán Trung Quốc, từ ngày 1/1/2007 đến 31/3/2022.
 - Dữ liệu được chia thành tập huấn luyện (3.500 mẫu) và tập kiểm tra (180 mẫu), sử dụng mốc ngày 22/6/2021 làm điểm chia.
- **Chuẩn bị và xử lý dữ liệu:**
 - Dữ liệu chuỗi thời gian gốc được xử lý trước bằng **ARIMA (Auto-Regressive Integrated Moving Average)** để ổn định chuỗi phi tuyến tính.
 - Dữ liệu được quan tâm là giá đóng cửa.

- **Kiến trúc mô hình:**

- **Preprocessing:** Dữ liệu qua ARIMA để biến đổi thành chuỗi dừng.
- **Pretraining:** Mô hình Attention-based CNN-LSTM thực hiện trích xuất đặc trưng qua Attention-based CNN (encoder) và khai thác chuỗi thời gian qua Bidirectional LSTM (decoder).
- **Fine-tuning:** Mô hình XGBoost được sử dụng để tinh chỉnh dự đoán cuối cùng, tận dụng các đặc trưng đã được trích xuất.

- **Thực hiện thực nghiệm:**

- Mô hình được triển khai trên GPU NVIDIA GTX 2070, với các thông số huấn luyện:
 - Epoch: 50
 - Batch size: 32
 - Dropout rate: 0,3
 - Optimizer: Adam với learning rate 0,01.
- Kỹ thuật "look-back window" được áp dụng để dự báo dựa trên chuỗi thời gian quá khứ.

- **Hiệu quả và so sánh:**

- Mô hình được đánh giá bằng các chỉ số: **MAE (Mean Absolute Error)**, **RMSE (Root Mean Square Error)**, **MAPE (Mean Absolute Percentage Error)** và **R²**.
- Kết quả cho thấy mô hình AttCLX đạt hiệu năng cao hơn so với các phương pháp truyền thống (như ARIMA, ARIMA-NN) và các mô hình hiện đại khác (như LSTM-KF, Transformer-KF).

2. Tập dữ liệu

Dữ liệu được sử dụng trong bài viết này đến từ bộ dữ liệu công khai miễn phí và mở tại Tushare (<https://www.tushare.pro/>). Ta lấy dữ liệu bằng cách gọi API.

Nghiên cứu thực nghiệm về giá cổ phiếu của Bank of China (601988.SH) trên thị trường chứng khoán Trung Quốc. Dữ liệu được chọn từ dữ liệu từ ngày 1 tháng 1 năm 2007 đến ngày 31 tháng 3 năm 2022, dữ liệu trong một ngày biểu thị một điểm của chuỗi. Bộ huấn luyện được lấy từ ngày 1/1/2017 đến ngày 22/06/2021 và bộ kiểm tra

được lấy vào ngày 22/06 /2021. Điều này có nghĩa là có 3500 mẫu đào tạo và 180 mẫu thử nghiệm. Kích thước của lô mỗi lần train là 32.

3. Giới thiệu cấu trúc mã nguồn

Các thư viện được dùng trong dự án:

1. Numpy v1.16.5
 2. Sklearn
 3. Statsmodels v0.10.1
 4. Pandas v0.25.1
 5. Tensorflow v2.3.1
 6. Xgboost v1.5.0
- Mã nguồn của dự án gồm 4 file chính: ARIMA.py, LSTM.py, XGBoost.py, Main.py.
 - Trước tiên chạy file ARIMA để thực hiện tiền xử lý dữ liệu bằng mô hình ARIMA

- Dữ liệu được lấy từ 601988.SH.csv, sau đó qua các bước tiền xử lý format dữ liệu và chia các tập training và tập test

```
data = pd.read_csv('./601988.SH.csv')
test_set2 = data.loc[3501:, :]
data.index = pd.to_datetime(data['trade_date'], format='%Y%m%d')
data = data.drop(['ts_code', 'trade_date'], axis=1)
data = pd.DataFrame(data, dtype=np.float64)

training_set = data.loc['2007-01-04':'2021-06-21', :] # 3501
test_set = data.loc['2021-06-22:', :] # 180
```

- Tính sai phân bậc 1 và bậc 2:

```

# First-order diff
training_set['diff_1'] = training_set['close'].diff(1)
plt.figure(figsize=(10, 6))
training_set['diff_1'].plot()
plt.title('First-order diff')
plt.xlabel('time', fontsize=12, verticalalignment='top')
plt.ylabel('diff_1', fontsize=14, horizontalalignment='center')
plt.show()

# Second-order diff
training_set['diff_2'] = training_set['diff_1'].diff(1)
plt.figure(figsize=(10, 6))
training_set['diff_2'].plot()
plt.title('Second-order diff')
plt.xlabel('time', fontsize=12, verticalalignment='top')
plt.ylabel('diff_2', fontsize=14, horizontalalignment='center')
plt.show()

```

- Tạo và training model ARIMA:

```

acf_pacf_plot(training_data_diff)

# order=(p,d,q)
model = sm.tsa.ARIMA(endog=training_set['close'], order=(2, 1, 0)).fit()

```

- Dự đoán giá dựa vào mô hình đã training:

```

history = [x for x in training_set['close']]
# print('history', type(history), history)
predictions = list()
# print('test_set.shape', test_set.shape[0])
for t in range(test_set.shape[0]):
    model1 = sm.tsa.ARIMA(history, order=(2, 1, 0))
    model_fit = model1.fit()
    yhat = model_fit.forecast()
    yhat = np.float(yhat[0])
    predictions.append(yhat)
    obs = test_set2.iloc[t, 5]
    # obs = np.float(obs)
    # print('obs', type(obs))
    history.append(obs)
    # print(test_set.index[t])
    # print(t+1, 'predicted=%f, expected=%f' % (yhat, obs))
#print('predictions', predictions)

```

- Chạy file LSTM cho các mô hình LSTM một chiều, hai chiều và nhiều chiều:

- Tiền xử lý dữ liệu:

```
yuan_data = pd.read_csv('./601988.SH.csv')
yuan_data.index = pd.to_datetime(yuan_data['trade_date'], format='%Y%m%d')
yuan_data = yuan_data.loc[:, ['open', 'high', 'low', 'close', 'amount']]

data = pd.read_csv('./ARIMA_residuals1.csv')
data.index = pd.to_datetime(data['trade_date'])
data = data.drop('trade_date', axis=1)
```

- Tạo chuỗi thời gian và tạo model LSTM:

```
X_train, y_train = data_split(training_set_scaled, n_timestamp)
yuan_X_train, yuan_y_train = data_split(yuan_training_set_scaled, n_timestamp)
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
yuan_X_train = yuan_X_train.reshape(yuan_X_train.shape[0], yuan_X_train.shape[1], 5)

X_test, y_test = data_split(testing_set_scaled, n_timestamp)

X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
yuan_X_test, yuan_y_test = data_split(yuan_testing_set_scaled, n_timestamp)
yuan_X_test = yuan_X_test.reshape(yuan_X_test.shape[0], yuan_X_test.shape[1], 5)

model, yuan_model = lstm(model_type, X_train, yuan_X_train)
```

- Training model LSTM:

```
history = model.fit(X_train, y_train,
                    batch_size=32,
                    epochs=n_epochs,
                    validation_data=(X_test, y_test),
                    validation_freq=1)
yuan_history = yuan_model.fit(yuan_X_train, yuan_y_train,
                              batch_size=32,
                              epochs=n_epochs,
                              validation_data=(yuan_X_test, yuan_y_test),
                              validation_freq=1)
```

- Chạy file XGBoost.py để sử dụng mô hình XGBoost

- Tiền xử lý dữ liệu đầu vào từ dữ liệu gốc và dữ liệu phần dư của ARIMA:

```

data = pd.read_csv('./601988.SH.csv')
data.index = pd.to_datetime(data['trade_date'], format='%Y%m%d')
data = data.loc[:, ['open', 'high', 'low', 'close', 'vol', 'amount']]
# data = pd.DataFrame(data, dtype=np.float64)
close = data.pop('close')
data.insert(5, 'close', close)
data1 = data.iloc[3501:, 5]
residuals = pd.read_csv('./ARIMA_residuals1.csv')
residuals.index = pd.to_datetime(residuals['trade_date'])
residuals.pop('trade_date')
merge_data = pd.merge(data, residuals, on='trade_date')

```

- Dự đoán phần dư bằng XGBoost:

```

train, test = prepare_data(merge_data, n_test=180, n_in=6, n_out=1)

y, yhat = walk_forward_validation(train, test)

```

- Cuối cùng chạy Main.py để chạy mô hình kết hợp ACNN-LSTM và XGBoost

- Hàm ACNN model:

```

def attention_model(INPUT_DIMS = 13, TIME_STEPS = 20, lstm_units = 64):
    inputs = Input(shape=(TIME_STEPS, INPUT_DIMS))

    x = Conv1D(filters=64, kernel_size=1, activation='relu')(inputs) # padding = 'same'
    x = Dropout(0.3)(x)

    # lstm_out = Bidirectional(LSTM(lstm_units, activation='relu'), name='bilstm')(x)
    lstm_out = Bidirectional(LSTM(lstm_units, return_sequences=True))(x)
    lstm_out = Dropout(0.3)(lstm_out)
    attention_mul = attention_3d_block(lstm_out)
    attention_mul = Flatten()(attention_mul)

    output = Dense(1, activation='sigmoid')(attention_mul)
    model = Model(inputs=[inputs], outputs=output)
    return model

```

- Xây dựng, biên dịch và training mô hình Attention-based:

```

m = attention_model(INPUT_DIMS=7)
m.summary()
adam = Adam(learning_rate=0.01)
m.compile(optimizer=adam, loss='mse')
history = m.fit([train_X], train_Y, epochs=50, batch_size=32, validation_split=0.1)
m.save("./stock_model.h5")
np.save("stock_normalize.npy", normalize)

```

- Dự đoán với mô hình đã huấn luyện

```
y_hat, y_test = PredictWithData(data2, data_yuan, name, 'stock_model.h5',7)
y_hat = np.array(y_hat, dtype='float64')
y_test = np.array(y_test, dtype='float64')
evaluation_metric(y_test,y_hat)
```

4. Kết quả thực nghiệm của tác giả

Sau khi tiến hành so sánh với các mô hình tiền đào tạo và tinh chỉnh khác nhau. Bảng III chứng minh rằng mô hình đề xuất vượt trội hơn các mô hình cơ sở khác.

TABLE III: Comparison with different pre-training and fine-tuning models

Pre-training	Fine-tuning	MSE	RMSE	MAE	R2
None	None	0.00057	0.02734	0.02368	0.74402
None	XGBoost	0.00031	0.01755	0.01223	0.82405
SL-LSTM	SL-LSTM	0.00045	0.02282	0.01960	0.79434
ML-LSTM	ML-LSTM	0.00031	0.01720	0.01265	0.82351
BiLSTM	BiLSTM	0.00027	0.01652	0.01201	0.84210
BiLSTM	XGBoost	0.00024	0.01605	0.01187	0.86301
CNN-BiLSTM	XGBoost	0.00022	0.01529	0.01145	0.87720
ACNN-BiLSTM	XGBoost	0.00020	0.01424	0.01126	0.88342

Theo bảng III cho thấy với pre-training là ACNN-BiLSTM và fine-tuning là XGBoost thì kết quả đầu ra của 4 hàm mất mát vượt trội hơn các mô hình khác, cụ thể với MSE là 0.0020, RMSE là 0.01424, MAE là 0.01126 và R2 là 0.88342.

Sau đó, tiến hành so sánh với các phương pháp hiện tại. Các phương pháp được so sánh bao gồm:

- ARIMA
- ARIMA-NN [17] cải thiện dự báo chuỗi thời gian bằng cách sử dụng mô hình ARIMA và mạng nơ-ron kết hợp.
- LSTM-KF, transformer-KF, TL-KF đề xuất Bộ lọc Kalman, cùng với LSTM và Transformer để dự đoán trữ lượng.

Bảng IV chứng minh rằng mạng nơ-ron được đề xuất hoạt động tốt hơn các phương pháp hiện tại.

TABLE IV: Results on different methods

Model	MAE	RMSE	MAPE	R2
ARIMA	0.00057	0.02734	0.02368	0.74402
ARIMA-NN	0.00052	0.02608	0.02350	0.75037
LSTM-KF	0.00047	0.02381	0.02192	0.76245
Transformer-KF	0.00037	0.01924	0.01525	0.80230
TL-KF	0.00033	0.01656	0.01372	0.81923
AttCLX	0.00020	0.01424	0.01126	0.88342

IV. Ứng dụng của bài báo

Bài báo đề xuất một mô hình kết hợp giữa ARIMA model , Attention CNN-LSTM và XGBoost để đưa ra mô hình dự đoán giá cổ phiếu dựa vào giá trị cổ phiếu trong quá khứ. Những ứng dụng điển hình của mô hình là:

- Áp dụng mô hình trong việc dự đoán thị trường chứng khoán : Cải thiện độ chính xác trong việc dự đoán giá cổ phiếu bằng cách khai thác sâu hơn vào giá trị dữ liệu lịch sử và chính xác hơn trong các mối quan hệ phi tuyến tính.
- Áp dụng trong lĩnh vực tài chính: Sử dụng trong ngân hàng cũng như những công ty chứng khoán giúp cải thiện hiệu quả dự báo, tăng hiệu suất trong các chiến dịch đầu tư và giao dịch.
- Bước đầu phát triển công nghệ AI trong lĩnh vực kinh tế: Thúc đẩy sự kết hợp của phương pháp học máy cùng các kỹ thuật dự đoán cổ điển như ARIMA để mở ra cơ hội kết hợp công nghệ AI trong lĩnh vực tài chính.

V. Kết luận

Bài báo đưa ra một mô hình gồm ARIMA model, Attention-based CNN-LSTM và XGBoost (gọi là AttCLX) để dự đoán giá cổ phiếu. Mô hình bài báo đã đưa ra được kết luận lại như sau:

- **Cải thiện hiệu quả của mô hình dự đoán truyền thống:** Bằng cách tích hợp AttCLX cùng ARIMA model để dự đoán giá cổ phiếu đã giúp cải thiện đáng kể độ chính xác so với các mô hình dự đoán cũ như ARIMA và các phương pháp học sâu đơn lẻ truyền thống như LSTM hay Transformer.

- **Khai thác thông tin đa chiều:** Mô hình đã tận dụng tốt các ưu điểm của mỗi thành phần.

- + **ARIMA:** Tiền xử lý dữ liệu và trích xuất các đặc trưng.

- + **Attention-base CNN:** Trích xuất các đặc trưng quan trọng từ chuỗi dữ liệu thời gian.

- + **LSTM:** Khai thác các mối quan hệ dài hạn trong chuỗi dữ liệu thời gian.

- + **XGBoost:** Tinh chỉnh kết quả cuối cùng, tăng khả năng mở rộng và tính linh hoạt và cải thiện độ chính xác của dự báo.

- **Ứng dụng thực tế trong tài chính:** Mô hình bài báo đề cập đã được thử nghiệm trên dữ liệu cổ phiếu từ thị trường chứng khoán Trung Quốc, kết quả nhận được đã cho số liệu tốt hơn khi so sánh với phương pháp truyền thống bằng cách so sánh các chỉ số hàm mất mát là MAE, RMSE, MAPE và R^2 .

VI. Phụ lục

Câu hỏi: tìm hai mã cổ phiếu, một mã cổ phiếu thỏa mãn độ bão hòa, một mã thì không thỏa mãn độ bão hòa và đánh giá?

1) Câu hỏi 1: tìm mã cổ phiếu thỏa mãn độ bão hòa và đánh giá?

Giá cổ phiếu của công ty cổ phần FPT (mã FPT) vượt qua bài kiểm tra Augmented Dickey-Fuller Test (ADF test) với sai phân bậc 2:

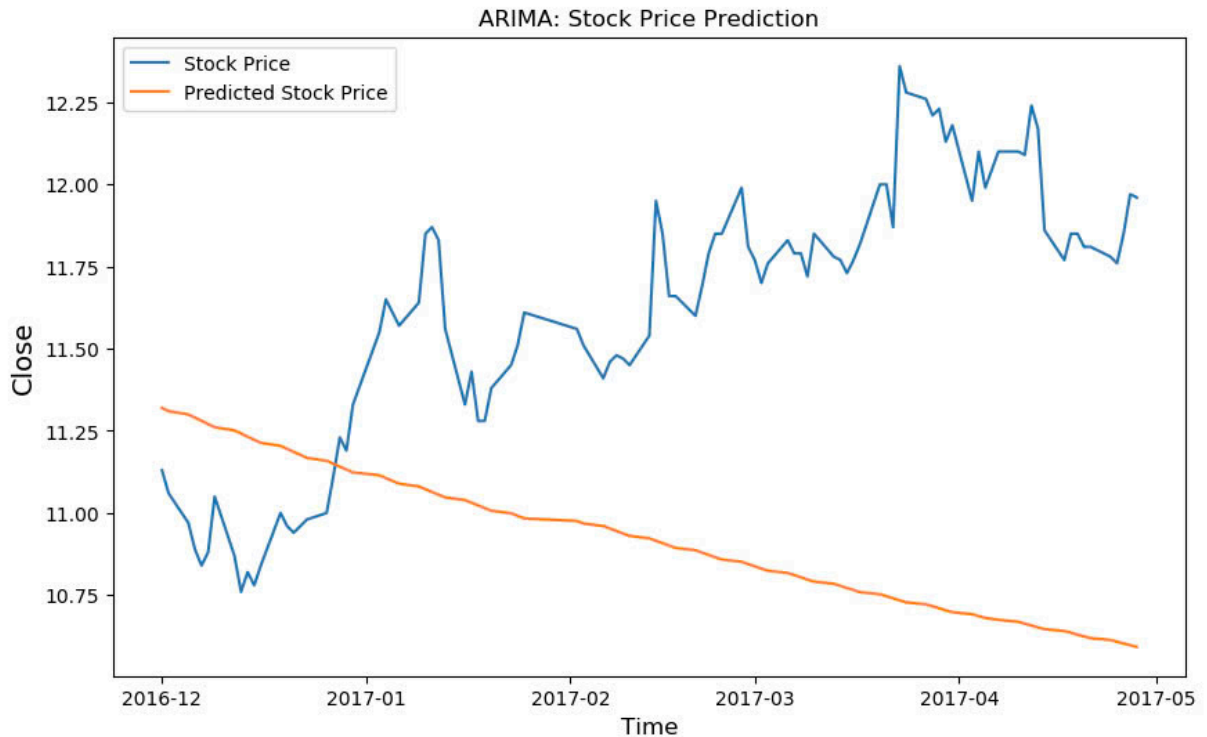
```
adf_test(np.diff(training_set["close"], n=2))
```

	value
Test Statistic Value	-11.71866
p-value	0.0
Lags Used	12
Number of Observations Used	635
Critical Value(1%)	-3.44069
Critical Value(5%)	-2.866102
Critical Value(10%)	-2.5692

với p-value = 0.0

và giá trị kiểm định là -11.72 nhỏ hơn tất cả các giá trị tới hạn ở các mức 1%, 5% và 10%.

Tuy nhiên giá trị dự đoán của mô hình ARIMA với các tham số ($p = 1$, $d=2$, $q = 0$) không tốt:



Mô hình ARIMA với các tham số ($p = 1, d=2, q = 0$) không dự đoán được xu hướng của giá cổ phiếu, nguyên nhân có thể là do chuỗi cần dự đoán cần thỏa mãn các điều kiện khác mà bài báo không nói tới.

2) Câu hỏi 2: tìm mã cổ phiếu không thỏa mãn độ bão hòa và đánh giá?

Giá cổ phiếu của chỉ số VNINDEX (mã VNINDEX) không vượt qua bài kiểm tra Augmented Dickey-Fuller Test (ADF test):

```

adf_test(np.diff(training_set["close"], n=0))
[18] ✓ 0.4s
...
value
Test Statistic Value    2.701945
p-value                 0.999087
Lags Used               31
Number of Observations Used  4193
Critical Value(1%)      -3.431911
Critical Value(5%)      -2.86223
Critical Value(10%)     -2.567137

```

với $p\text{-value} = 0.99$

và giá trị kiểm định là 2.7 lớn hơn tất cả các giá trị tới hạn ở các mức 1%, 5% và 10%.

Do đó kết quả dự đoán của mô hình ARIMA không tốt

